mpitbR: An R Package for Calculating Multidimensional Poverty Indices

by Ignacio Girela

Abstract Multidimensional poverty measurement is a vital tool for assessing the deprivations and well-being of people across different dimensions, such as health, education, and living standards. However, existing packages for multidimensional poverty measurement have some limitations, such as ignoring the complex survey design of microdata. In this paper, we present mpitbR, a package for calculating multidimensional poverty indices based on the Alkire-Foster measurement approach, which accounts for the survey design, and offers various options and features for users. This package is the R version of the Stata mpitb package, which reproduces the workflow of the global Multidimensional Poverty Index developed by the United Nations Development Programme and Oxford University. The package provides functions for estimating the Alkire-Foster measures, as well as for analyzing poverty changes over time. The usage of the main functions and features of the mpitbR package is described and illustrated with an application over a synthetic data that has a typical household survey design.

1 Introduction

The first goal of the 2030 Agenda for Sustainable Development ambitiously postulates *Ending poverty in all its forms everywhere*. This call to global action recognizes poverty as not merely a lack of income but as a complex phenomenon with multiple dimensions that affect individuals' well-being.

To capture these diverse aspects of poverty, the last two decades have witnessed the development of various methodologies for multidimensional poverty measurement. The Alkire and Foster (2011) (AF) method, employing a *dual-cut-off-counting* approach, has gained wide adoption in both academic research and policy-making spheres due to its simplicity and rigorous theoretical foundation. Notably, the AF method allows disaggregation into various partial indices (AF measures), which serve as powerful tools for policy analysis and design (Alkire, 2021). The global Multidimensional Poverty Index (MPI), published annually by the United Nations Development Programme (UNDP) and the Oxford Poverty and Human Development Initiative (OPHI), exemplifies the worldwide relevance of the AF method (UNDP and OPHI, 2022). Furthermore, there is an increasing number of countries that have built their national MPIs based on the AF framework tailored to their local contexts in order to monitor and evaluate their poverty reduction strategies (OPHI).

Although the AF method has become a prominent framework for measuring multidimensional poverty, practical challenges arise in calculating these measures. On one hand, the construction of an MPI naturally hinges on the careful consideration and justification of its dimensions, indicators, and parameters, such as the weights and poverty lines, to ensure its accurate reflection of poverty's complexities within a specific context. On the other hand and very importantly, poverty measurement often relies on household survey data, and ignoring the complex survey design in poverty analysis can lead to biased estimates and erroneous statistical inferences, particularly when comparing poverty levels across subgroups or over time.

Therefore, reliable multidimensional poverty estimation requires tools that facilitate easy parameter management and account for complex survey designs. Existing Stata packages like *mpitb* (Suppa, 2023) address this need by providing functionalities to estimate AF measures and their associated standard errors. This Stata package provides a user-friendly way to replicate the widely-used global MPI methodology, establishing a solid guideline for researchers and practitioners in multidimensional poverty analysis. However, users without Stata expertise or who prefer the R programming environment lack a comparable solution in R.

The main contribution of this paper is to introduce the **mpitbR** package (Girela, 2025), a novel and valuable tool for multidimensional poverty measurement and analysis in R. **mpitbR** package builds upon the global MPI workflow, replicating the functionality of the aforementioned Stata package. Furthermore, unlike existing R packages, namely **MPI** (Kukiattikun and Chainarong, 2022) and **mpindex** (Abdulsamad, 2024), this package accounts for the complex design of household surveys by means of the **survey** package methods (Lumley, 2004, 2010, 2024). This ensures reliable estimation of AF measures, as well as their associated standard errors and confidence intervals, which are crucial for robust statistical inference. By addressing the survey design, **mpitbR** fills a critical gap in the R ecosystem for poverty researchers. The paper demonstrates the practical functionality of **mpitbR** with examples of common multidimensional poverty analysis exercises using real-world household survey data.

The rest of this paper is organized as follows. Section 2 briefly introduces the AF method and the

MPI calculation. Section 3 describes the **mpitbR** package and its main functions. Section 4 illustrates the usage of the package with some examples and applications. Section 5 concludes with a summary and provides some suggestions for further extensions of the package.

2 Measuring multidimensional poverty

Alkire and Foster (2011) proposed a multidimensional poverty measurement method (AF method) based on a *dual-cut-off-counting-approach* for the identification and aggregation of the poor. Constructing an MPI based on the AF method consists of the following steps (Alkire et al., 2015):

1. Determine a set of dimensions of poverty, $\mathcal{D}$, that are considered relevant for human development in a specific context (e.g., the global MPI chooses dimensions of health, education, and living standards, but other dimensions can be chosen depending on the context and goals).
2. Select d indicators that represent deprivations in each dimension (e.g., child mortality and undernutrition are the two indicators that represent the health dimension in the global MPI).
3. Assign weights to each dimension and indicator, reflecting their relative importance, where w_j represents the weight of the j-th indicator for $j = 1, \ldots, d$ such that $\sum_{j=1}^{d} w_j = 1$. In practice, an equal nested weighting scheme is used: dimensions are weighted equally as well as each indicator within the dimension.
4. Set the indicators deprivation cut-offs, which define the minimum level of achievement required to be considered non-deprived in each indicator (for example, in the global MPI, a person is considered deprived in Years of Schooling indicator if she has less than six years of formal education).
5. Apply the deprivation cut-offs vector to each of the n observations (individuals or households) and build the $n \times d$ deprivation matrix, $\mathbf{g^0}$. Each element of this matrix, $\left[\mathbf{g^0}\right]_{ij}$, is a binary variable. If $\left[\mathbf{g^0}\right]_{ij} = 1$, the i-th observation is deprived in indicator j, and the opposite if $\left[\mathbf{g^0}\right]_{ij} = 0$.
6. Build the weighted deprivation matrix, $\bar{\mathbf{g}}^{\mathbf{0}}$, assigning the corresponding weight to each indicator (i.e., $\bar{\mathbf{g}}^{\mathbf{0}} = \mathbf{g^0} \times \mathbf{diag}(w_1, \ldots, w_d)$) and calculate the deprivations score for each observation, c_i, which is the weighted sum of the deprivations, $c_i = \sum_{j=1}^{d} w_j \, g_{ij}^0$.
7. Identify who is poor by setting a unique poverty cut-off, k, meaning the minimum proportion of weighted deprivations that an individual needs to experience to be considered multidimensional poor. This cut-off is compared with the deprivations score, c_i. Therefore, if $c_i \geq k$, the i-th observation in the sample is classified as multidimensional poor (e.g., the global MPI uses a cut-off, k, of 33.3% or $1/3$, meaning that people are considered to be poor if they experience deprivations in one-third or more of the weighted indicators).
8. Censor data of the non-poor to obtain the so-called censored (weighted) deprivation matrix, $\bar{\mathbf{g}}^{\mathbf{0}}(k)$, and censored deprivations scores, $c(k)$, where $c_i(k) = c_i$ if $c_i \geq k$, and $c_i(k) = 0$ otherwise.
9. Compute the M_0 by taking the mean of the censored deprivations score, $c(k)$:

$$M_0 = \frac{1}{n} \sum_{i=1}^{n} c_i(k) \tag{1}$$

The M_0 measure, also known as *Adjusted Headcount Ratio*, is sometimes equated with the MPI. Hereafter, we will use both terms interchangeably. Furthermore, the M_0 measure can be re-expressed as a function of other partial measures, which we refer to as AF measures. For instance,

- The M_0 measure can be decomposed into two key components: the *Incidence* (H) and the *Intensity* (A) of poverty. Formally, this is represented as:

$$M_0 = \frac{q}{n} \times \frac{1}{q} \sum_{i=1}^{n} c_i(k) = H \times A$$

  Here, q is the number of people identified as multidimensional poor, $H = \frac{q}{n}$ is the proportion of multidimensionally poor people (the incidence), and $A = \frac{1}{q} \sum_{i=1}^{n} c_i(k)$ represents the average proportion of weighted deprivations experienced by the poor (the intensity).

- The MPI can also be calculated as the weighted sum of the *Censored Indicators Headcounts Ratios*, $h_j(k)$, for $j = 1, \ldots, d$. This is clearer by rearranging the order of aggregation in the original M_0 equation (1). This is,

$$M_0 = \frac{1}{n}\sum_{i=1}^{n} c_i(k) = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{d} \bar{g}_{ij}^0(k) \tag{2}$$

$$= \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{d} w_j g_{ij}^0(k) = \sum_{j=1}^{d} w_j \frac{1}{n}\sum_{i=1}^{n} g_{ij}^0(k) \tag{3}$$

$$= \sum_{j=1}^{d} w_j h_j(k) \tag{4}$$

  Then, $h_j(k)$ represents the proportion of people who are deprived in indicator *j and* are multidimensional poor.
- From the uncensored deprivations matrix, $\mathbf{g^0}$, we can obtain their uncensored versions, or the *Uncensored Headcount Ratio* for each indicator $j = 1, \ldots, d$, denoted as h_j, which is the proportion of people deprived in indicator j. The comparison between the censored and uncensored indicators headcount ratios is insightful for targeted interventions (Alkire, 2021).
- We can also decompose the MPI by different population subgroups, such as age, regions, sex, etc.. Formally,

$$M_0 = \sum_{l=1}^{L} \phi^l M_0^l$$

  where ϕ^l is the population share of the l-th level for $l = 1, \ldots, L$, and L is the total number of levels in the subgroup. For instance, the population subgroup "sex" has two levels of analysis: male and female, encoded as $l = 1, 2$. This property also applies to other AF measures (H, A, h_j, $h_j(k)$).
- The *absolute* and *proportional contribution* of each indicator to poverty are usually reported, calculated as $w_j\, h_j(k)$ and $w_j\, h_j(k)/M_0$, respectively.

Changes over time

Alkire et al. (2017) proposed four measures to assess pro-poor multidimensional poverty reduction between two periods of time using repeated cross-sectional data. The following measures are explained using the Adjusted Headcount Ratio, M_0, however, all the changes-over-time measures can be extended to the aforementioned AF measures (H, A, h_j, $h_j(k)$).

Let t^1 and t^2 denote the initial and final period, respectively. Then, $M_0^{t^1}$ and $M_0^{t^2}$ are their corresponding Adjusted Headcount Ratios. Note that for comparability purposes, these two poverty measures must have the same set of parameters (indicators, weights, deprivations and poverty cut-offs).

We define the *absolute rate of change* (Δ) as the difference in M_0 between the final t^2 and the initial period t^1:

$$\Delta(M_0) = M_0^{t^2} - M_0^{t^1}$$

The *relative rate of change* (δ) is defined as the difference in M_0, expressed as a percentage of the initial poverty level t^1, i.e.,

$$\delta(M_0) = \frac{M_0^{t^2} - M_0^{t^1}}{M_0^{t^1}} \times 100$$

On the other hand, the annualized versions of the absolute and relative rates of change are used in order to compare changes over time across countries with different periods of reference.

The *annualized absolute rate of change* ($\bar{\Delta}$) is the absolute rate of change (as defined above) divided by the difference between the two time periods:

$$\bar{\Delta}(M_0) = \frac{M_0^{t^2} - M_0^{t^1}}{(t^2 - t^1)}$$

Finally, the *annualized relative rate of change* ($\bar{\delta}$) is defined as the compound annual rate of reduction in M_0 between the initial and the final periods, i.e.,

$$\bar{\delta}(M_0) = \left[\left(\frac{M_0^{t^2}}{M_0^{t^1}} \right)^{\frac{1}{t^2 - t^1}} - 1 \right]$$

An explanation of the usage of the main functions for estimating AF measures and their changes over time using the **mpitbR** package is provided in the following section.

3 Overview of mpitbR package

mpitbR adapts the Stata *mpitb* framework for R users, ensuring consistency with the original workflow. The package offers two main functions, `mpitb.set` and `mpitb.est`, that mirror the functionality of the key Stata subcommands. This means that users familiar with Stata will find similar input parameters and output formats in **mpitbR**, facilitating a smooth transition between the two packages. The estimation of AF measures is based on these core functions. In addition, **mpitbR** includes built-in R functions, such as `coef`, `confint`, and `summary`, to allow users to examine the results (i.e., estimated coefficients, confidence intervals, and summary statistics of the AF measures). As mentioned above, **mpitbR** mainly depends upon the R package **survey** to account for the complex survey design during estimation.

3.1 `mpitb.set` function

This function, mirroring the `mpitb set` subcommand in Stata, serves as the setup step for **mpitbR**. It prepares the data for AF measure estimation by specifying the deprivation indicators and incorporating the survey design.

The usage and input arguments of function `mpitb.set` are summarized as follows:

```
mpitb.set(data, indicators, ...,
          name = "unnamed", desc = "desc.")
```

Arguments:

- `data`: A dataset containing household survey information. To incorporate survey design (e.g., sampling weights, strata and primary sampling units), use the `svydesign` function from the **survey** package and provide a "survey.design2"-class object to this argument. If `data` is a regular data frame, **mpitbR** coerces it to a "survey.design2"-class object. However, this process assumes a simple random sampling design, which is often inappropriate for household surveys and can lead to biased results. The function does issue a warning message to alert users about this assumption and the potential need to specify a more complex survey design if their data warrants it.
- `indicators`: This argument specifies the deprivation indicators (columns) in `data` that define the MPI dimensions. It can be provided in two formats:
  - a list: This is useful for defining the set of dimensions and their corresponding indicators. Each element in the list represents a dimension with a character vector containing its indicator names. This format facilitates automatic calculation of equal nested weights during estimation (up to 10 dimensions allowed). Under this format, users can also specify weights for unequal weighting of dimensions.
  - a character vector: This is more convenient when indicators are not grouped by dimensions. In this case, users need to specify weights manually during the AF measure estimation.

Details for the arguments:

- The deprivation indicators specified in `data` should be binary variables (0/1), i.e., it is assumed that users are working with the deprivation matrix, $\mathbf{g}^0$.
- The data should not contain missing values in these indicator columns. While the **survey** package can handle missing values for point estimates, it is crucial to note that it cannot calculate standard errors and confidence intervals when missing data are present.

Optional arguments:

- `name`: a short project name for identification (maximum 10 characters).

- `desc`: A brief description of the project setting.

Output:

The function returns an object of class `"mpitb_set"` containing:

- Survey design data (from the **survey** package).
- Names of the deprivation indicators.
- Project name and description (if provided).

3.2 `mpitb.est` function

Building upon the information from `mpitb.set`, the `mpitb.est` function performs the core calculations for AF measure estimation. Here users define all the parameters and the desired measures to be calculated.

```
mpitb.est(set, klist = NULL, weights = "equal",
          measures = c("M0", "H", "A"),
          indmeasures = c("hd", "hdk", "actb", "pctb"),
          indklist = NULL,  over = NULL,  ...,
          cotyear = NULL,  tvar = NULL,
          cotmeasures = c("M0", "H", "A", "hd", "hdk"),
          ann = FALSE,  cotklist = NULL,
          cotoptions = "total",
          noraw = FALSE,  nooverall = FALSE,
          level = 0.95,
          multicore = getOption("mpitb.multicore"),
          verbose = TRUE)
```

Main arguments:

- `set` (required): An object of class `"mpitb_set"` created by the `mpitb.set` function. This object encapsulates the previously prepared data and settings for estimation.
- `klist` (required): A numeric vector of poverty cut-offs (k) as percentages (between 1 and 100) for multidimensional poverty measurement. Default values are not specified.
- `weights`: This argument defines the weighting scheme for the deprivation indicators. It can be provided in two formats:
  - a character (default): If `weights = "equal"`, it automatically calculates equal nested weights.
  - a numeric vector: In order to provide the weights manually, users can input a numeric vector of values between 0 and 1 such that all sum up to 1. Their values should match the order of the indicators in the `set` argument.
- `measures`: A character vector that allows selecting the specific AF measures to be calculated (M_0, H, and A). By default, `mpitb.est` calculates each one of these measures (`c("M0","H","A")`).
- `indmeasures`: A character vector that allows selecting the (censored and uncensored) headcount ratio of each indicator and their contribution (absolute and proportional). By default, `mpitb.est` calculates each one of these measures (`c("hd","hdk","actb","pctb")`).

Optional arguments:

- `indklist`: A numeric vector with poverty cut-offs (numbers between 1 and 100) for the indicator-specific measures.
- `over`: A character vector specifying the column names of the population subgroups (e.g., sex) in `data`, for which AF measures will be estimated across their respective levels of analysis (i.e., male and female).

Further details:

- The `measures` and `indmeasures` arguments allow users to avoid unnecessary estimations. If any of these arguments is `NULL`, `mpitb.est()` skips these measures. For instance, if `measures = c("M0","H","A")` and `indmeasures = NULL`, only the M_0, H, and A will be estimated.

- If `indmeasures` is specified, but `indklist` is not, `indklist` defaults to the poverty cut-offs provided in `klist`.
- The `svyciprop` function from the **survey** package is used to estimate confidence intervals, treating the measures as proportions. This function employs the 'logit' method, which is often preferred for proportions as it helps to keep the confidence intervals within the valid 0 to 1 range. The 'logit' method involves fitting a logistic regression model and calculating a Wald-type interval on the log-odds scale, which is then transformed back to the probability scale.

Changes over time analysis

mpitbR allows users to analyze how multidimensional poverty changes across different time periods in the survey data. This requires specifying a few additional arguments:

- `tvar`: A character vector indicating the column name in the data that identifies the time period or survey round. This argument is always required for changes over time analysis.
- `cotyear` (optional): A character vector indicating the column name in the data that contains the year information for each survey round (decimal values allowed). This is required for annualized measures.
- `cotmeasures` (optional): A character vector specifying the particular AF measures to estimate for changes over time. By default, all AF measures are calculated. However, users can save computational time by specifying preferred measures. For example, if `cotmeasures = c("M0","H,","A")` only changes in M_0, H and A will be calculated.
- `cotklist` (optional): A numeric vector specifying poverty cut-offs for the changes over time analysis. If not provided, it defaults to the cut-offs provided in `klist`.
- `cotoptions` (optional): This argument is used only if users have more than two survey rounds. It determines how changes are estimated:
  - `"total"` (default): Estimates changes over the entire observation period.
  - `"insequence"`: Estimates year-to-year changes.

Further details:

The standard errors of changes-over-time measures are estimated using the Delta method (detailed in `svycontrast()` function from **survey** package).

Additional control arguments

- `ann` and `noraw` (optional): Logical values that control whether to estimate annualized and non-annualized changes-over-time measures. By default, both are `FALSE` (non-annualized).
  - If `cotyear` is provided (year information), `ann` is automatically set to `TRUE` (annualized).
  - If `cotyear` has values but `ann` is `FALSE`, only non-annualized measures are estimated.
  - If only annualized measures are desired, users can set `noraw` to `TRUE` in order to avoid non-annualized calculations.
- `nooverall` (optional): A logical value that controls whether to estimate poverty measures for the overall data (e.g., national level). By default, it is `FALSE` (estimates overall). Setting `nooverall` to `TRUE` skips overall estimates.
- `level` (optional): Specifies the confidence level for estimated confidence intervals (defaults to 95%). Users can adjust this value.
- `multicore` (optional): Enables parallel computations for all measures and poverty cut-offs. This argument uses the "Forking" method. Hence, this option is only available on Unix-like operating systems.
- `verbose` (optional): A logical value that controls whether to print messages or not. By default, it is `TRUE`.

Table 1: Comparison of R Packages for Multidimensional Poverty Calculation

Feature	mpitbR	mpindex	MPI
Estimate AF disaggregated measures	Yes	Yes	Yes
Include changes over time analysis	Yes	No	No
Account for survey design	Yes	No	No
Flexibility	Medium (mirrors UNPD methodology)	High (permits building the deprivations matrix from data)	Medium (works directly with the deprivations matrix)
Ease of use	Medium (few functions with large number of arguments)	Low (MPI specifications from external files (.txt, .csv, .json))	High (few functions with small number of arguments)
Include R build-in functions for analyzing results	Yes	No	No

3.3 Other functions

The output of `mpitb.est()` function mirrors the structure of the Stata package. It is a two-element list where each element is a `data.frame` containing the estimated values. These data frames follow the same format as the Stata package output.

Then, users can apply functions such as `coef`, `confint`, and `summary`. Since the output data may contain multiple AF measures, by cut-offs and levels of the subgroups and possibly by different time periods, it is recommended to filter the data for specific analyses with these functions.

`summary` function performs a t-test on the estimates, inheriting the confidence level in the `level` argument of `mpitb.est` function. This function is particularly helpful for the changes-over-time estimates, where the user can infer if there has been a statistically significant reduction in multidimensional poverty between two time periods. However, when conducting changes-over-time analysis, the `summary` function requires filtering the output to a single AF measure, one change-over-time measure, and one poverty cut-off to perform a distinct statistical test.

The next section provides examples of how to implement these functions in practice using a household survey.

3.4 Comparison with other packages

While **mpindex** and **MPI** also calculate AF-based multidimensional poverty indices, including their disaggregation properties, a crucial advantage of **mpitbR** lies in the fact that it accounts for the household survey design. This allows for rigorous statistical inference, a feature conspicuously absent in **mpindex** and **MPI**. The ability to perform such analysis becomes particularly crucial for understanding poverty dynamics over time, a functionality uniquely offered within the **mpitbR** ecosystem.

Due to its dependence on the **survey** package functions and its aim to mirror the Stata `mpitb` workflow, **mpitbR** suffers from a degree of inflexibility. However, this is largely compensated by its consistency with the official calculation methodology used by the United Nations. Conversely, **mpindex** and **MPI** offer greater flexibility in their implementation. In addition, **mpindex** allows for building the deprivations matrix from data within its workflow, unlike the other two packages which require a predefined deprivations matrix for the calculations.

However, the **mpindex** package can be harder to implement since all the specifications of the MPI calculation (dimensions, indicators, weights, and poverty cut-offs) have to be set out from the R environment (using .txt, .csv, or .json files). In contrast, **MPI** and **mpitbR** packages are fairly easier to use, although **mpitbR** demands a steeper learning curve due to the large number of arguments.

Last but not least, **mpitbR** leverages R's built-in functions for analyzing results, such as `coef()`, `confint()`, and `summary()`, a feature that is totally absent in the other two packages. Table 1 summarizes these comparisons.

4 How to use the package

To bridge the gap between theory and practice, this section provides illustrative examples aiming to showcase the application of the **mpitbR** package for common exercises encountered by researchers in real-world multidimensional poverty analysis. We use the `syn_cdta` dataset, a synthetic dataset with a household survey structure, also used in the original `mpitb` Stata package examples by Suppa (2023). This allows for a direct comparison between the Stata `mpitb` framework and the **mpitbR** package.

Table 2: A synthetic household survey data

d_nutr	d_cm	d_satt	d_educ	d_elct	d_sani	d_wtr	d_hsg	d_ckfl	d_asst	area	region	stratum	psu	weight	year	t
0	0	1	0	0	0	0	1	0	0	1	4	401	401000	1	2010	1
0	0	0	0	1	0	0	1	1	1	1	1	104	104003	1	2010	1
0	0	0	0	1	0	0	0	0	0	1	20	2002	2002005	1	2010	1
0	0	0	0	0	0	0	1	0	0	1	20	2004	2004002	1	2010	1
0	0	1	0	1	0	0	1	0	0	1	18	1805	1805000	1	2010	1
0	0	1	0	1	1	0	1	0	1	0	18	1803	1803001	1	2010	1

4.1 The syn_cdta Dataset

The `syn_cdta` dataset included in the `mpitbR` package comprises 15,000 observations, representing individuals, and includes 17 variables. These variables can be categorized as follows:

- **Ten Binary Indicator Variables:** These variables represent different dimensions typically measured in the Multidimensional Poverty Index (MPI). Examples might include Nutrition, Child Mortality, School Attendance, and so on. For detailed descriptions of the specific MPI dimensions included, please refer to the UNDP and OPHI (2022).
- **Two Subgroup Variables:** The dataset includes two categorical variables, "area" and "region", that can be used for subgroup analysis.
- **Survey design variables:** Three additional variables, "psu", "stratum", and "weight", define the complex survey design.
- **Survey Round and Year:** Two additional variables, "t" and "year" respectively, identify the survey round and year of data collection for each observation.

For a detailed list of variables refer to Table 2.

4.2 Example 1: Cross-Sectional Estimations for a Single Country

First of all, we load the required packages.

```
library(survey)
library(mpitbR)
```

The analysis is restricted to the first survey round (i.e., subsetting data with `t == 1`), and observations with missing values in some deprivation indicators are removed, as the **survey** package methods do not support missing values for standard error calculations.

```
# Keep observation from the first survey round
first_round <- subset(syn_cdta, t == 1)
# Drop NA in deprivation indicators columns
first_round <- na.omit(first_round)
```

Next, we define the survey design using the `svydesign` function from the **survey** package, considering the primary sampling units (`psu`), weights (`weight`), and strata (`stratum`) information in the data.

```
# Define survey design
svydata <- svydesign(id=~psu, weights = ~weight, strata = ~stratum, data = first_round)
```

To specify the MPI estimation settings, we use the `mpitb.set` function. The MPI typically considers three core dimensions: Health (`hl`), Education (`ed`), and Living Standards (`ls`). Here, we define indicators for each dimension:

- Health: `d_nutr` (deprivation in Nutrition indicator) and `d_cm` (child mortality indicator)
- Education: `d_satt` (deprivation in School Attendance indicator) and `d_educ` (deprivation in Years of Schooling indicator)
- Living Standards: `d_elct` (deprivation in Electricity Access indicator), `d_sani` (deprivation in Improved Sanitation indicator), `d_wtr` (deprivation in Drinkable Water Access indicator), `d_hsg` (deprivation in Housing Conditions indicator), `d_ckfl` (deprivation in Cooking Fuel indicator), and `d_asst` (deprivation in Assets Ownership indicator)

```
# Define the dimensions of poverty with their corresponding indicators
indicators <- list(hl = c("d_nutr","d_cm"),
                   ed = c("d_satt","d_educ"),
                   ls = c("d_elct","d_sani","d_wtr","d_hsg","d_ckfl","d_asst"))
# Set the multidimensional poverty project
set <- mpitb.set(svydata, indicators = indicators,
                         name = "trial01", desc = "pref. spec")
```

Finally, we define a short name (`"trial01"`) and description (`"pref. spec"`) for our project using `mpitb.set`.

For this first round, assume we want to estimate all the AF measures. These include the aggregate measures (M_0, H, and A) and the indicator-specific measures (h_d, $h_d(k)$, $actb$, and $pctb$). We prefer an equal-nested weighting scheme (equal weights for all dimensions and equal indicator weights within dimensions). Additionally, for each measure, we set three poverty cut-offs: 20%, 33%, and 50%. Finally, we want to calculate the disaggregated measures by subnational regions and living areas (urban and rural), accounting for the complex design of the survey.

This information is specified in the `mpitb.est` function as follows:

```
est <- mpitb.est(set = set, klist = c(20, 33, 50),
                         weights = "equal",
                 measures = c("M0","H","A"),
                 indmeasures = c("hd", "hdk", "actb", "pctb"),
                 over = c("area","region"))
```

```
#>         ****** SPECIFICATION ******
#> Call:
#> mpitb.est.mpitb_set(set = set, klist = c(20, 33, 50), weights = "equal",
#>     measures = c("M0", "H", "A"), indmeasures = c("hd", "hdk",
#>         "actb", "pctb"), over = c("area", "region"))
#> Name:  trial01
#> Weighting scheme:  equal
#> Description:  pref. spec
#> ____________________
#>
#> Dimension 1:  hl 0.333                                  (d_nutr, d_cm)
#> Dimension 2:  ed 0.333                                (d_satt, d_educ)
#> Dimension 3:  ls 0.333 (d_elct, d_sani, d_wtr, d_hsg, d_ckfl, d_asst)
#> ____________________
#>
#> Indicator 1:   d_nutr 0.1667
#> Indicator 2:     d_cm 0.1667
#> Indicator 3:   d_satt 0.1667
#> Indicator 4:   d_educ 0.1667
#> Indicator 5:   d_elct 0.0556
#> Indicator 6:   d_sani 0.0556
#> Indicator 7:    d_wtr 0.0556
#> Indicator 8:    d_hsg 0.0556
#> Indicator 9:   d_ckfl 0.0556
#> Indicator 10:  d_asst 0.0556
#>
#>         ****** ESTIMATION ******
#> ____________________
#> Partial AF measures: ' M0 H A ' under estimation... DONE
#>
#> ____________________
#> Indicator-specific measures: ' hd hdk actb pctb ' under estimation... DONE
#>
#>         ****** RESULTS ******
#> ____________________
#> Parameters
#> Subgroups:  3
#> Poverty cut-offs (k):  3
#>
#> *Notes:
```

```
#>   Confidence level: 95 %
#>   Parallel estimations:  FALSE
```

The `mpitb_set`-class object is provided in the `set` argument. The poverty cut-offs are specified in `klist` and the population subgroups in `over`. The equal-nested weighting scheme is specified in the `weights` argument by passing the character string `"equal"`. The `measures` argument specifies the aggregate measures, and `indmeasures` specifies the indicator-specific measures we want to estimate. However, by default, all these measures and equal-nested weights are calculated, so these arguments can be omitted in this case.

Users can verify the specification of their project using the messages returned by `mpitb.est`. These messages report:

1. The function call, verifying correct argument assignment.
2. The dimensions with their assigned indicators and corresponding weights.
3. Which measures are being estimated.
4. The estimation parameters (number of poverty cut-offs and subgroups).
5. Other features such as the confidence level and whether parallel estimation has been used.

All these messages can be suppressed using `verbose = FALSE` in `mpitb.est` function.

Recall that the `mpitb.est` function returns a two-element list (of class `"mpitb_est"`), where each element is a data frame containing all the estimates. The first element, named `"lframe"`, includes all the cross-sectional estimates for each level of analysis. The second element, named `"cotframe"`, contains all the changes over time measures for each level of analysis. In this first example, since we are not analyzing changes over time, `"cotframe"` will be `NULL`.

For this instance, the first rows of the `lframe` data frame can be examined using `head(est$lframe)`.

```
head(est$lframe)
```

```
#>           b          se        ll        ul subg    loa measure ctype  k
#> 1 0.2169686 0.002363434 0.2123628 0.2216461  nat    nat      M0   lev 20
#> 2 0.2165175 0.003493254 0.2097356 0.2234567    0   area      M0   lev 20
#> 3 0.2172704 0.002873528 0.2116799 0.2229667    1   area      M0   lev 20
#> 4 0.2229081 0.011585856 0.2009836 0.2464864    1 region      M0   lev 20
#> 5 0.2111428 0.010886855 0.1905533 0.2333157    2 region      M0   lev 20
#> 6 0.2302469 0.013024076 0.2056679 0.2568137    3 region      M0   lev 20
#>   indicator
#> 1      <NA>
#> 2      <NA>
#> 3      <NA>
#> 4      <NA>
#> 5      <NA>
#> 6      <NA>
```

Each row in this data frame represents an estimate for a specific measure and population subgroup. In this example, with three poverty cut-offs, two subgroups, and seven measures (three aggregate and four indicator-specific), we have a total of 2507 estimates. This structure closely mirrors the output of the Stata package. The first columns include the most important features such as the point estimate (b), its corresponding standard error (se) that accounts for the complex survey design, and the lower and upper confidence limits (ll and ul) calculated by default at a 95% confidence level. The remaining columns specify the corresponding AF measure (measure), its corresponding indicator (if applicable), its poverty cut-off (k), its subgroup level of analysis (loa) and its subgroup category (subg).

Since the elements of the list are data frames, users can easily subset the 'lframe' or 'cotframe' elements to examine point estimates or confidence intervals of specific subgroups and measures of interest. For instance, suppose we want to see the confidence intervals for the incidence (H) measure for the `"area"` subgroups at the 33% poverty cut-off. The code for this would be:

```
confint(subset(est$lframe, measure == "H" & loa == "area" & k == 33))
```

```
#>   Subgroup Level of analysis Cut-off Lower Bound (95%) Upper Bound (95%)
#> 1        0              area      33             0.325             0.358
#> 2        1              area      33             0.317             0.345
```

Finally, `summary` can be used to calculate a t-test statistic for each point estimate in the data frames, along with the corresponding p-value and significance level. This is particularly useful for inferring whether a change-over-time measure is statistically different from zero. We will provide an example of this functionality below.

4.3 Example 2: Saving Time with Package Options

Efficiently prioritizing estimates can save time when dealing with numerous parameters like poverty cut-offs and population subgroups. As observed in the previous example, the number of estimates can increase rapidly even with a small number of parameters.

One way to optimize estimations is to consider the characteristics of the measures themselves. For instance, deprivation scores, c_i and $c_i(k)$, reach a finite number of values for fixed weights and cut-offs. Therefore, it is important to be aware of the included cut-off values to avoid redundant estimations. This becomes even more relevant when incorporating population subgroups, which significantly contribute to the growth in estimates.

The **mpitbR** package provides options to control which measures are estimated and the poverty cut-offs used. The `klist` argument specifies cut-offs for aggregate measures, while `indklist` allows defining a separate list for indicator-specific measures.

In the following code, we replicate the previous example with a few adjustments to demonstrate these time-saving options:

```
mpitb.est(set = set, klist = c(20, 33, 50),
          weights = "equal", measures = c("M0"),
          indmeasures = c("hd", "hdk"), indklist = c(33),
          over = c("area","region"), nooverall = TRUE)
```

Here, we focus on the M_0 measure and exclude the H and A measures, along with contribution measures. Additionally, we specify a different cut-off value (33%) for indicator-specific measures and avoid national level estimates. As a result, the number of estimates is significantly reduced compared to the previous example (946 vs. 2507).

4.4 Example 3: Specify Alternative Weighting Schemes

In the previous examples, we assumed equal weights across dimensions and indicators (equal-nested weights). This can be achieved by passing the indicators grouped in a list to the `mpitb.set` function and setting `weights = "equal"` in the `mpitb.est` function (the default).

An alternative approach is to specify weights as a numeric vector in the `weights` argument. This can be more efficient for defining complex weighting schemes. For example, the previously employed equal nested weights, using a numerical vector, can be expressed as follows:

```
mpitb.est(set = set, klist = c(33),
          measures = c("M0"), indmeasures = NULL,
          weights = c(1/6, 1/6, 1/6, 1/6,
                      1/18, 1/18, 1/18, 1/18, 1/18, 1/18))
```

We can also assign different weights to each dimension. For instance, if we assign a 50% weight to the Health dimension and a 25% weight to each of the remaining dimensions, with equal weights within each dimension, the weights vector will be:

```
mpitb.est(set = set, klist = c(33),
          measures = c("M0"), indmeasures = NULL,
          weights = c(1/4, 1/4, 1/8, 1/8,
                      1/24, 1/24, 1/24, 1/24, 1/24, 1/24))
```

This approach allows users to readily assess the impact of adding/dropping indicators, merging indicators, or using different deprivation thresholds. Specifying alternative weighting schemes is often used during MPI construction. This approach serves two purposes: exploring parsimony of the measure and comparing orderings under different weighting scenarios (Santos and Villatoro, 2016; Alkire et al., 2022).

The following code demonstrates how to modify the MPI calculations by removing the electricity indicator (`d_elct`) from the analysis.

```
indicators <- list(hl = c("d_nutr","d_cm"),
                   ed = c("d_satt","d_educ"),
                   ls = c("d_sani","d_wtr","d_hsg","d_ckfl","d_asst"))
```

```
set <- mpitb.set(syn_cdta, indicators = indicators,
                 name = "trial02", desc = "w/o electricity")

est <- mpitb.est(set = set, klist = c(20, 33, 50), weights = "equal",
                 over = c("area","region"))
```

4.5 Example 4: Changes over time analysis

In order to analyse pro-poor changes over time, we will use the entire data set to define our survey design with the difference that we have to specify which column indexes the survey rounds in the `tvar` argument of the `mpitb.est` function.

```
# Drop NA in deprivation indicators columns
syn_cdta <- na.omit(syn_cdta)
# Define survey design
svydata <- svydesign(id=~psu, weights = ~weight, strata = ~stratum, data = syn_cdta)

# Define list of indicators
indicators <- list(hl = c("d_nutr","d_cm"),
                   ed = c("d_satt","d_educ"),
                   ls = c("d_elct","d_sani","d_wtr","d_hsg","d_ckfl","d_asst"))

# Specify MPI estimates setting
set <- mpitb.set(svydata, indicators = indicators,
                 name = "trial01", desc = "Estimate changes over time")
```

We are interested in estimating AF measures for each population subgroup across both years and assessing whether poverty has decreased significantly. The following code estimates M_0, H, and A measures for each year, along with measures of poverty changes over time, both at the regional and national levels. Setting `indmeasure = NULL` avoids estimating indicator-specific measures by levels. We restrict the analysis of changes over time to the AF measure calculated by each year: `c(M0, H, A)`. Remind that for estimating annualized measures, we need to specify the year column in `cotyear` argument.

```
# Estimation
est <- mpitb.est(set, klist = c(1, 33, 50),
                 indmeasures = NULL,
                 cotmeasures = c("M0","A","H"),
                 over = c("region"), tvar = "t", cotyear = "year")

#>         ****** SPECIFICATION ******
#> Call:
#> mpitb.est.mpitb_set(set = set, klist = c(1, 33, 50), indmeasures = NULL,
#>     over = c("region"), cotyear = "year", tvar = "t", cotmeasures = c("M0",
#>         "A", "H"))
#> Name:  trial01
#> Weighting scheme:  equal
#> Description:  Estimate changes over time
#> ____________________
#>
#> Dimension 1:  hl 0.333                                (d_nutr, d_cm)
#> Dimension 2:  ed 0.333                               (d_satt, d_educ)
#> Dimension 3:  ls 0.333 (d_elct, d_sani, d_wtr, d_hsg, d_ckfl, d_asst)
#> ____________________
#>
#> Indicator 1:   d_nutr 0.1667
#> Indicator 2:     d_cm 0.1667
#> Indicator 3:   d_satt 0.1667
#> Indicator 4:   d_educ 0.1667
#> Indicator 5:   d_elct 0.0556
#> Indicator 6:   d_sani 0.0556
#> Indicator 7:    d_wtr 0.0556
#> Indicator 8:    d_hsg 0.0556
```

```
#> Indicator 9:   d_ckfl 0.0556
#> Indicator 10:  d_asst 0.0556
#>
#>         ****** ESTIMATION ******
#> ____________________
#> Partial AF measures: ' M0 H A ' under estimation... DONE
#>
#> ____________________
#> Estimate changes over time over ' M0 A H ' measures... DONE
#>
#>         ****** RESULTS ******
#> ____________________
#> Parameters
#> Number of time periods:  2
#> Subgroups:  2
#> Poverty cut-offs (k):  3
#>
#> *Notes:
#>   Confidence level: 95 %
#>   Parallel estimations:  FALSE
```

Suppose we want to examine if the number of poor people has decreased overall for a poverty cut-off of 33%. We can observe the coefficients of the incidence of poverty at the national level.

```
coef(subset(est$lframe, k == 33 & measure == "H" & loa == "nat"))
```

```
#>   Subgroup Level of analysis Cut-off Coefficient
#> 1      nat               nat      33       0.335
#> 2      nat               nat      33       0.231
```

In this case, we observe that the national Incidence of poverty, H, decreased from 33.5% to 23.1%. Examining the confidence intervals, if they do not overlap, it suggests a significant reduction of poverty (assuming a 95% confidence level by default). To do so, we use the `confint` function.

```
confint(subset(est$lframe, k == 33 & measure == "H" & loa == "nat"))
```

```
#>   Subgroup Level of analysis Cut-off Lower Bound (95%) Upper Bound (95%)
#> 1      nat               nat      33             0.324             0.346
#> 2      nat               nat      33             0.222             0.240
```

If we want to investigate if this national-level tendency holds at each regional level, we can use the `summary` function. This allows us to explore the statistical significance of the changes-over-time measures (here, we focus on non-annualized absolute and relative measures, as annualized measures are typically used for cross-country comparisons).

```
summary(subset(est$cotframe, k == 33 &
                 measure == "H" & loa == "region" &
                 ann == 0 & ctype == "abs"))
```

```
#>
#> Measure: H
#> Coefficients:
#>   Non-annualized  Absolute change over time measures:
#>  Poverty cut-off: 33 %
#>           Estimate  Std.Err t-value Pr(>|t|)
#> 1.region  -0.08364  0.03035  -2.755 0.005862 **
#> 10.region -0.13474  0.03205  -4.205 2.62e-05 ***
#> 11.region -0.08914  0.03375  -2.641 0.008259 **
#> 12.region -0.07353  0.03510  -2.095 0.036194 *
#> 13.region -0.08019  0.03333  -2.406 0.016137 *
#> 14.region -0.09188  0.03130  -2.935 0.003334 **
#> 15.region -0.11548  0.03929  -2.939 0.003290 **
#> 16.region -0.14710  0.03152  -4.667 3.06e-06 ***
```

```
#> 17.region -0.09310  0.03159  -2.948 0.003203 **
#> 18.region -0.03016  0.03137  -0.961 0.336415
#> 19.region -0.15718  0.02635  -5.965 2.44e-09 ***
#> 2.region  -0.10215  0.02797  -3.653 0.000259 ***
#> 20.region -0.10135  0.02623  -3.864 0.000112 ***
#> 3.region  -0.13947  0.03469  -4.020 5.81e-05 ***
#> 4.region  -0.07307  0.03677  -1.987 0.046893 *
#> 5.region  -0.13783  0.03228  -4.270 1.95e-05 ***
#> 6.region  -0.10612  0.02689  -3.946 7.94e-05 ***
#> 7.region  -0.07524  0.03829  -1.965 0.049445 *
#> 8.region  -0.10993  0.03931  -2.797 0.005164 **
#> 9.region  -0.16716  0.03465  -4.824 1.41e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

summary(subset(est$cotframe, k == 33 &
                measure == "H" & loa == "region" &
                ann == 0 & ctype == "rel"))

#>
#> Measure: H
#> Coefficients:
#>   Non-annualized  Relative change over time measures:
#>  Poverty cut-off: 33 %
#>           Estimate Std.Err t-value Pr(>|t|)
#> 1.region   -22.887   7.277  -3.145 0.001660 **
#> 10.region  -35.796   6.540  -5.474 4.41e-08 ***
#> 11.region  -28.953   9.485  -3.052 0.002270 **
#> 12.region  -22.727   9.480  -2.397 0.016510 *
#> 13.region  -25.838   8.915  -2.898 0.003751 **
#> 14.region  -25.986   7.566  -3.434 0.000594 ***
#> 15.region  -36.113  10.161  -3.554 0.000380 ***
#> 16.region  -43.751   6.901  -6.340 2.30e-10 ***
#> 17.region  -29.186   8.550  -3.413 0.000641 ***
#> 18.region  -10.463  10.395  -1.007 0.314166
#> 19.region  -43.284   5.593  -7.739 1.00e-14 ***
#> 2.region   -32.303   6.917  -4.670 3.01e-06 ***
#> 20.region  -31.130   7.023  -4.433 9.31e-06 ***
#> 3.region   -39.850   7.541  -5.285 1.26e-07 ***
#> 4.region   -24.271  10.657  -2.277 0.022760 *
#> 5.region   -40.165   7.565  -5.309 1.10e-07 ***
#> 6.region   -28.963   6.378  -4.541 5.60e-06 ***
#> 7.region   -23.366  10.622  -2.200 0.027816 *
#> 8.region   -32.528   9.030  -3.602 0.000315 ***
#> 9.region   -42.857   6.797  -6.305 2.88e-10 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

These results suggest that there have been statistically significant reductions in the incidence of poverty between the two time periods (with different levels of confidence) in each region, with the exception of region 18, where we cannot conclude that the number of multidimensional poor people has been reduced.

5 Summary

Multidimensional Poverty Indices (MPIs) are crucial tools for measuring poverty beyond sheer income, considering multiple deprivations individuals face. This paper has introduced the **mpitbR** package, a user-friendly toolkit for estimating MPIs based on the Alkire-Foster method. This package offers researchers and practitioners a comprehensive framework for multidimensional poverty measurement.

mpitbR is tailored to the specific needs of the global MPI workflow, reflecting the functionality of the *mpitb* Stata package. This includes data requirements, standard outputs, and relevant analysis tools. The unified output structure, while mirroring the original package, constrains the development

of straightforward built-in plot functions for visualizing the estimated results, forcing a trade-off between simplicity and the extensive conditional logic required to adapt generic functions to various Alkire-Foster measures included in the output. However, **mpitbR** holds significant potential for further development.

Future versions of the package will incorporate functions for robustness analysis (to assess the sensitivity of the results to specific choices, such as poverty cut-offs or weights), interlinkages analysis (to explore which combinations of deprivations are more prevalent among the poor) and visualizing results (to explore data and communicate findings, maximizing user convenience despite the output complexity).

Additionally, implementing functions for panel data analysis within the Alkire-Foster framework would be valuable for longitudinal studies. Including novel methods for analyzing inequality among the poor within the AF method would also be highly beneficial.

Ultimately, the future development of **mpitbR** will also be guided by user needs, ongoing research advancements, and available resources. In sum, by providing a powerful and adaptable toolkit, **mpitbR** actively contributes to the field of multidimensional poverty measurement.

6 Acknowledgements

I would like to express my sincere thanks both to Rodrigo García Arancibia and José Vargas, my mentors, for their invaluable guidance, feedback, and encouragement throughout the development of this package. I also wish to acknowledge Nicolai Suppa for his generous support and enthusiasm in adapting his Stata package to R users. Without his constructive suggestions this package would not have been possible. Finally, I would like to thank every instructor from the OPHI Summer School 2022, for their thorough lectures on the Alkire-Foster method for multidimensional poverty measurement and the provision of the Stata source code which mainly motivated this project.

References

B. Abdulsamad. *mpindex: Multidimensional Poverty Index (MPI)*, 2024. URL https://CRAN.R-project.org/package=mpindex. R package version 0.2.1. [p95]

S. Alkire. *Multidimensional Poverty Measures as Policy Tools*, chapter Dimensions of Poverty: Measurement, Epistemic Injustices, Activism, pages 197–214. Philosophy and Poverty. Springer, 2021. ISBN 9783030317102. [p95, 97]

S. Alkire and J. Foster. Counting and multidimensional poverty measurement. *Journal of Public Economics*, 95(7-8):476–487, 08 2011. doi: 10.1016/j.jpubeco.2010.11.006. URL http://dx.doi.org/10.1016/j.jpubeco.2010.11.006. [p95, 96]

S. Alkire, J. Foster, S. Seth, M. E. Santos, J. M. Roche, and P. Ballon. *Multidimensional Poverty Measurement and Analysis*. Oxford University Press, 2015. URL https://doi.org/10.1093/acprof:oso/9780199689491.001.0001. [p96]

S. Alkire, J. M. Roche, and A. Vaz. Changes Over Time in Multidimensional Poverty: Methodology and results for 34 countries. *World Development*, 94:232–249, 2017. ISSN 0305-750X. doi: 10.1016/j.worlddev.2017.01.011. URL https://www.sciencedirect.com/science/article/pii/S0305750X17300256. [p97]

S. Alkire, U. Kanagaratnam, R. Nogales, and N. Suppa. Revising the global multidimensional poverty index: Empirical insights and robustness. *Review of Income and Wealth*, 68(S2), 03 2022. doi: 10.1111/roiw.12573. URL http://dx.doi.org/10.1111/roiw.12573. [p105]

I. Girela. *mpitbR: Calculate Alkire-Foster Multidimensional Poverty Measures*, 2025. URL https://CRAN.R-project.org/package=mpitbR. R package version 1.0.1. [p95]

K. Kukiattikun and C. Chainarong. *MPI: Computation of Multidimensional Poverty Index (MPI)*, 2022. URL https://CRAN.R-project.org/package=MPI. R package version 0.1.0. [p95]

T. Lumley. Analysis of Complex Survey Samples. *Journal of Statistical Software*, 9(8):1–19, 2004. doi: 10.18637/jss.v009.i08. [p95]

T. Lumley. *Complex Surveys: A Guide to Analysis Using R: A Guide to Analysis Using R*. John Wiley and Sons, 2010. [p95]

T. Lumley. *survey: Analysis of Complex Survey Samples*, 2024. URL https://CRAN.R-project.org/package=survey. R package version 4.4. [p95]

OPHI. National MPI and policy. https://ophi.org.uk/national-mpi-and-policy. URL https://ophi.org.uk/national-mpi-and-policy. Accessed: 2024-06-15. [p95]

M. E. Santos and P. Villatoro. A multidimensional poverty index for Latin America. *Review of Income and Wealth*, 64(1):52–82, 11 2016. doi: 10.1111/roiw.12275. URL http://dx.doi.org/10.1111/roiw.12275. [p105]

N. Suppa. mpitb: A toolbox for multidimensional poverty indices. *The Stata Journal*, 23(3):625–657, 2023. doi: 10.1177/1536867X231195286. URL https://doi.org/10.1177/1536867X231195286. [p95, 101]

UNDP and OPHI. 2022 Global Multidimensional Poverty Index (MPI). *UNDP (United Nations Development Programme)*, 2022. URL https://hdr.undp.org/content/2022-global-multidimensional-poverty-index-mpi. [p95, 102]

Ignacio Girela
CONICET, Universidad Nacional de Córdoba
Facultad de Ciencias Económicas, Bv. Enrique Barros s/n
Córdoba, Argentina
https://github.com/girelaignacio
ORCiD: 0000-0003-3297-3854
ignacio.girela@unc.edu.ar